

introduction to patterns and nesting pattern

introduction

when designing parallel algorithms, we often encounter recurring problems that require similar structural solutions

a **parallel pattern** is a recurring combination of task distribution and data access that solves a specific problem in parallel algorithm design

patterns provide a universal “vocabulary” for algorithm design

instead of focusing immediately on low-level implementation details, we can describe a program’s structure using these standard terms

patterns are universal concepts

they can be implemented in any parallel programming system (**e.g.**, CUDA, OpenMP, MPI)

using a pattern means identifying a specific problem and applying a precise solution structure that has been proven to work for that scenario

classification of patterns

to analyze programs effectively, we categorize patterns based on what they manage - either the flow of control or the organization of data

serial/parallel control patterns

these define how tasks are ordered and executed **e.g.** Sequence, Fork-Join, Map

serial/parallel data management patterns

these deal with how data is allocated, shared and accessed **e.g.** Partitioning, Geometric Decomposition

structural patterns

such as the **Nesting Pattern**, which describes the overall composition of the algorithm

other patterns

programming model support for patterns

il nesting pattern è un pattern compositazionale → non descrive un algoritmo specifico ma definisce la capacità di comporre gerarchicamente altri pattern in un'applicazione

a fundamental **compositional pattern** that provides the ability to hierarchically compose other patterns, allowing for modular and scalable algorithm design

the Nesting Pattern allows a “Task Block” in an algorithm to be composed of other patterns

just as a programming language allows functions to call other functions

this pattern appears in both serial and parallel algorithms

nesting pattern

we can use **Pattern Diagrams** to visualize this hierarchy

in these diagrams, a generic “Task Block” represents a unit of work

the Nesting Pattern dictates that any specific Task Block can be expanded or replaced by another pattern

provided that the new pattern maintains the same input/output interfaces and respects existing dependencies

forma dinamica di annidamento → invece di avere una struttura statica definita dal programmatore, la ricorsione crea nuovi livelli di annidamento a runtime

a single green block in the hierarchy represents a high-level task

this block can be expanded internally to reveal a more complex internal structure

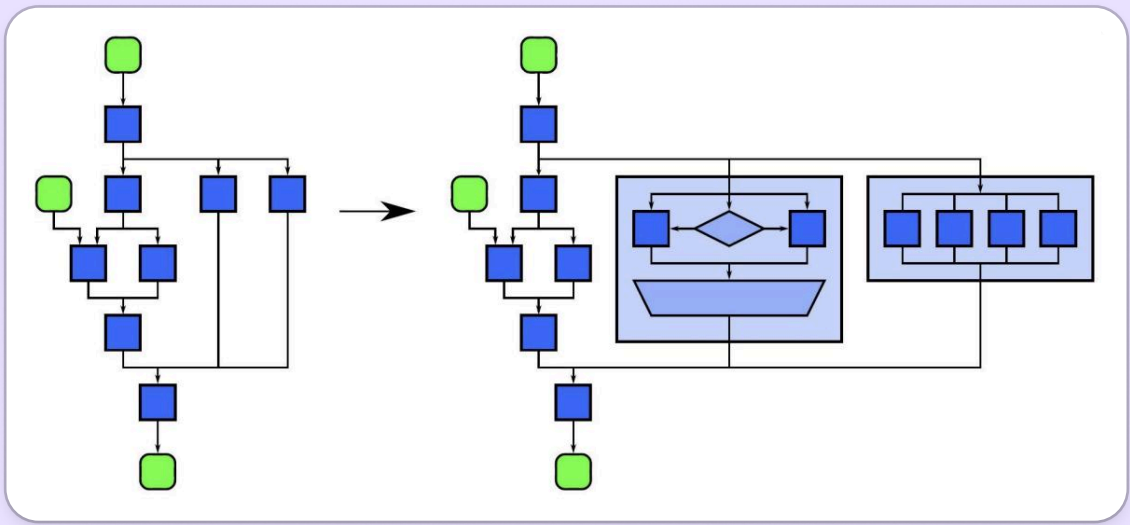
selection (branching/if-else)

parallel loop

this hierarchy is used to describe parallelism

if two blocks are on parallel branches of the hierarchy, they can run simultaneously

furthermore, because of nesting, a block running in parallel with another can *itself* contain further internal parallelism



Come funziona?

1. **Gerarchia dei Blocchi:** L'idea fondamentale è che qualsiasi “blocco di task” (un'unità di lavoro generica in un diagramma) può essere sostituito da un intero altro pattern, purché le caratteristiche di input/output e le dipendenze siano compatibili. ☞ ☞ ☞
2. **Componibilità:** Questo permette di costruire algoritmi complessi combinando pattern più semplici. Ad esempio, non sei costretto a scegliere tra usare una *Map* o una *Fork-Join*; puoi annidarli uno dentro l'altro.